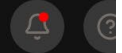


# Container list



Containers

Q Search...



▶ Start

□ Stop

/ Kill

↺ Restart

⏸ Pause

▶ Resu



Name ↓↑

State ↓↑ Filter 

Quick Actions

Stack ↓↑

Image ↓↑



app-backend-1

running



app

app-backend



app-frontend-1

running



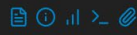
app

app-frontend



app-postgres-1

running



app

postgres:16-trixie



docker-caddy-1

running



docker

lucaslorentz/caddy-docker-proxy:ci-alpine



docker-portainer-1

running



docker

portainer/portainer-ce:latest



docker-webhook-1

running



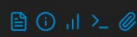
docker

docker-webhook



keycloak-keycloak-1

running



keycloak

quay.io/keycloak/keycloak:22.0.3



keycloak-postgresql-1

running



keycloak

postgres:16



langflow

running



app

langflowai/langflow:latest



openstreetmap

running



app

overv/openstreetmap-tile-server

| Feature         | WireGuard                                                                                        | OpenVPN                                                                                    | Headscale (Tailscale)                                                                                                  |
|-----------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Speed / Latency |  Extremely Fast |  Moderate |  Extremely Fast                     |
| Codebase Size   | ~4,000 lines (Easy to audit)                                                                     | ~100,000+ lines (Complex)                                                                  | Leverages WireGuard                                                                                                    |
| Topology        | Peer-to-Peer or Hub-and-Spoke                                                                    | Client-Server                                                                              | Full Mesh (Direct peer connections)                                                                                    |
| NAT Traversal   | Requires manual port forwarding                                                                  | Requires manual port forwarding                                                            |  Automatic (STUN/ICE hole-punching) |
| Best Used For   | High-performance, fixed point-to-point connections.                                              | Complex enterprise environments with legacy firewalls.                                     | Seamlessly connecting multiple roaming devices without port forwarding.                                                |

## The Strategy

1. **Public Layer:** Only your frontend container retains its **caddy labels** on safepath.duckdns.org. Caddy will route global traffic to the frontend only.
2. **Private Layer (The VPN):** Drop all Caddy routing labels from your backend, OSM, and Langflow services.
3. **Internal Routing:** Your frontend app runs in the user's browser, meaning it still needs to talk to an API endpoint. Instead of routing browser API traffic out to a public sub-domain, you will connect your **VPN container directly to the caddy network**. Caddy will then use local IP/domain routing restricted to the VPN interface, or you can query services directly via the VPN network mesh.

## Step 1: Clean up docker-compose.yml

Remove the public Caddy labels from backend and openstreetmap. They will still live on the caddy network so they can talk to each other internally, but they will no longer have domain endpoints exposed to the wild.

```
services:
# -----
# SafePath Vue.js frontend -> STILL PUBLIC
# -----
frontend:
  build: ./frontend
  command: npm run dev
  env_file:
    - .env
  depends_on:
    - backend
  networks:
    - caddy
  labels:
    caddy: safepath.duckdns.org
    caddy.reverse_proxy: "{{upstreams 8080}}"

# -----
# Backend API -> LOCKED DOWN (Labels Removed)
# -----
backend:
  build: ./backend
  env_file:
    - .env
  networks:
    caddy:
      aliases:
        - backend.local
  environment:
    - ENV=production

# -----
# OpenStreetMap tile server -> LOCKED DOWN (Labels Removed)
# -----
openstreetmap:
  image: overv/openstreetmap-tile-server
  container_name: openstreetmap
  volumes:
    - berlin-data:/data/database/
  command: run
  restart: always
  environment:
    - ALLOW_CORS=enabled
    - SSL_POLICY=none
  networks:
    - caddy

osm-import:
  image: overv/openstreetmap-tile-server
  profiles: [ "setup" ]
  volumes:
    - berlin-data:/data/database/
  environment:
    - DOWNLOAD_PBF=https://download.geofabrik.de/europe/germany/berlin-latest.osm.pbf
    - DOWNLOAD_POLY=https://download.geofabrik.de/europe/germany/berlin.poly
  command: import

volumes:
  berlin-data:
    external: true
    name: senior-project-safepath_berlin-data

networks:
  caddy:
    external: true
```

## Step 2: Clean up docker-compose-langflow.yml

Remove the Caddy labels from Langflow so it is no longer exposed to the public web via langflow.safepath.duckdns.org.

```
YAML
services:
  langflow:
    image: langflowai/langflow:latest
    container_name: langflow
    environment:
      - LANGFLOW_CONFIG_DIR=/app/langflow
      - LANGFLOW_DATABASE_URL=postgresql://langflow:langflow@postgres:5432/langflow
      - DO_NOT_TRACK=True
      - LANGFLOW_SECRET_KEY=${LANGFLOW_SECRET_KEY}
      - LANGFLOW_CORS_ORIGINS=https://safepath.duckdns.org,http://localhost:8080
      - LANGFLOW_CORS_ALLOW_METHODS=GET,POST,PUT,PATCH,DELETE,OPTIONS
      - LANGFLOW_CORS_ALLOW_HEADERS=Content-Type,Authorization,x-api-key
      - LANGFLOW_CORS_ALLOW_CREDENTIALS=True
    volumes:
      - langflow-data:/app/langflow
    restart: unless-stopped
    env_file:
      - .env
    depends_on:
      - postgres
    networks:
      - langflow-net
      - caddy # Kept so backend or VPN-connected services can reach it inside Docker

  postgres:
    image: postgres:16-trixie
    environment:
      POSTGRES_USER: langflow
      POSTGRES_PASSWORD: langflow
      POSTGRES_DB: langflow
    networks:
      - langflow-net
    volumes:
      - langflow-postgres:/var/lib/postgresql/data

volumes:
  langflow-data:
  langflow-postgres:

networks:
  langflow-net:
    driver: bridge
  caddy:
    external: true
```

## Step 3: Implement WireGuard - Create a new docker-compose-infra.yml

To access these hidden components from your local development machine or phone, add a simple WireGuard service container to your server infrastructure stack. This configuration places WireGuard on the same external caddy network, allowing you to access any container by its internal container name or container IP once connected.

Add this service to a standalone core infrastructure file:

YAML



```
services:
  wg-easy:
    environment:
      # Change this to your server's public WAN IP or public domain
      - WG_HOST=your-server-public-ip-or-domain
      - PASSWORD=ChooseAStrongAdminPasswordHere
      # Matches the subnet assigned inside the VPN tunnel
      - WG_DEFAULT_ADDRESS=10.8.0.x
      - WG_DEFAULT_DNS=1.1.1.1
      - WG_ALLOWED_IPS=10.8.0.0/24, 172.16.0.0/12 # Allows access to both VPN and in
    image: ghcr.io/wg-easy/wg-easy
    container_name: wg-easy
    volumes:
      - ./wireguard:/etc/wireguard
    ports:
      - "51820:51820/udp" # WireGuard VPN tunnel port
      - "51821:51821/tcp" # Web UI port (Protect this!)
    restart: unless-stopped
    capabilities:
      - NET_ADMIN
      - SYS_MODULE
    sysctls:
      - net.ipv4.conf.all.src_valid_mark=1
      - net.ipv4.ip_forward=1
    networks:
      - caddy
```

#### Step 4: Update deploy.sh

Bash



```
#!/bin/sh
cd /app

git fetch origin prod
git reset --hard origin/prod
git clean -fd

echo "Building core security & infrastructure stack"
docker compose -f docker-compose-infra.yml up -d --build

echo "Building project stack"
docker compose up -d --build

echo "Building langflow stack"
docker compose -f docker-compose-langflow.yml up -d --build
```